

Bachelorarbeit im Studiengang Computerlinguistik (B.A.)
an der Heinrich-Heine-Universität Düsseldorf



Optimizing RAG Pipelines for Small Language Models

Evaluating Document Preprocessing, Chunking, and Query Expansion
on Structured Administrative Data

vorgelegt von:

Daniel Gelderblom
Otto-Hahn-Straße 118
40591 Düsseldorf
Daniel.Gelderblom@hhu.de
Matrikelnummer: 3131057

Abgabedatum: 19. September 2026
Erstgutachterin: Prof. Dr. Wiebke Petersen
Zweitgutachter: David Arps

Contents

1	Introduction	1
2	Data	2
2.1	Documents	2
2.2	Evaluation Dataset	3
3	RAG pipeline	4
3.1	Code Architecture	4
3.2	Document preprocessing	5
3.3	Chunking	7
3.4	Indexing and Retrieval	9
3.5	Query expansion	9
3.6	Reranking	9
3.7	Generation	10
3.8	Left-out Methods	10
4	Experiments	11
4.1	Evaluation Setup	11
4.2	Pipeline Configuration	12
4.3	Document Preprocessing Experiment	12
4.4	Grid Search for Chunking Hyperparameters	12
4.5	Reranking Threshold Experiment	13
4.6	Query Expansion Experiment	13
4.7	Main Experiment: Preprocessor and Chunker Comparison	13
5	Results	14
5.1	Preprocessing choices	14
5.2	Chunking Parameters	14
5.3	Context Size and Reranking Threshold	15
5.4	Query Expansion	16
5.5	Main Experiment	17
6	Discussion	18
6.1	Preprocessing methods	18
6.2	Chunking Parameters	18
6.3	Reranking Threshold	19
6.4	Query Expansion	20
6.5	Chunking and Preprocessing Baselines	20
6.6	Detailed Analysis of Markdown Chunking Strategies	20
6.6.1	Paragraph Chunking and Whole Table Chunking	20
6.6.2	Character Chunking and Split Table Chunking	21
6.7	Detailed Analysis of Text Chunking Strategies	23
6.7.1	Paragraph Chunking	23
6.7.2	Character Chunking	23
6.7.3	Dynamic Token Chunking	24
6.7.4	LLM Chunking	24
6.7.5	Max-Min Chunking	27

6.8	Negative Rejection	28
6.9	Evaluation Analysis	29
7	Limitations and Future Research	31
8	Conclusion	32

List of Figures

1	The RAG pipeline	3
---	----------------------------	---

List of Tables

1	Representative Sample Question-Answer Pairs from the Evaluation Dataset	4
2	Sample Question-Answer Pairs from the Negative Rejection Dataset . . .	5
3	Comparison of Document Preprocessors	14
4	Grid Search Results for Chunking Hyperparameters	15
5	Comparison of Reranking Thresholds for each Preprocessor/Chunker Combination with Average Tokens per Chunk	16
6	Evaluation of RAG Pipeline Answer Correctness Across Query Processors.	17
7	Evaluation of RAG Pipeline Across Preprocessing and Chunking Strategies Across All Metrics.	18
8	Examples of the SLM failing to utilize split table content	22
9	Information Extraction and Refusal Errors in SLM Outputs for Paragraph Chunking on text files	23
10	Examples of Successful Extraction versus Missing Context Retrieval for Character Chunking	25
11	Examples of Successful Extraction versus Information Loss in Dynamic Token Chunking	26
12	Comparison of Chunk Splitting Examples for Max-Min Chunking	28
13	Examples of Hallucinations and Their Context Anchors for Negative Rejection	29
14	Evaluation Analysis	30

1 Introduction

Since the publication of OpenAI’s GPT-3 (Brown et al. 2020), Large Language Models (LLMs) have gained immense importance. They are used in many contexts, for example code generation, summarization or data analysis. A major problem with LLMs, however, is their tendency to hallucinate. Especially in contexts like user queries on specific documents, factual correctness is vital. Over time, Retrieval Augmented Generation (RAG) emerged as a solution to this problem.

Before the rise of transformer-based LLMs, language models had received all of their knowledge during training. With transformer models, Guu et al. (2020) first used retrieval from a knowledge base together with generation during training, thus not needing to train the model on all knowledge. Karpukhin et al. (2020) improved dense retrieval (retrieval based on similarity scores of text embeddings), making it better than keyword-based retrieval and thus completing the foundation for RAG. RAG (Lewis et al. 2020) combines a (most often dense) retriever with a generative model. The retriever takes documents from a knowledge base relevant to a query and feeds them to the text generator to answer the query.

As context windows have increased to one million tokens, the need for RAG systems for recent proprietary LLMs, while still present, has decreased. However, these LLMs cannot serve every use-case. When data is confidential, privacy is crucial, and only a locally running language model can be considered. In these settings, resource constraints often apply. Thus, only Small Language Models (SLMs) are an option here. SLMs have smaller context windows (262,144 tokens for Qwen3.5-2B from 2026). Far more importantly, due to their small size, they suffer much more from lost-in-the-middle problems. Lost-in-the-middle describes the phenomenon where transformer-based language models fail drastically when the answer to a question is placed in the middle of a long prompt (N. F. Liu et al. 2024). Thus, performance often drops far before reaching the context limit. For SLMs around 8B parameters, this happens as early as 4000 to 8000 input tokens (Leng et al. 2024). This means that only a few short documents can be part of the prompt, increasing the importance of a well-designed RAG pipeline, and especially of the retrieval step. Improving this step, specifically through document preprocessing, chunking, and query expansion, is what this work focuses on.

Preprocessing is the conversion of documents into a form that is easy for the LLM to understand. Chunking is the technique employed to split the documents, which are all larger than 8000 tokens, into smaller pieces. Query expansion means transforming the query into a form that is easier for the retriever to handle. Thus, the first two parts of the pipeline are only done once for all documents in the offline phase, while query expansion is done for every user input in the online phase (Figure 1).

The task at hand is to create a chat-bot to answer administrative questions from students for the Department of Computational Linguistics. The long-term goal is to have this chat-bot run locally on a small GPU. Therefore, only SLMs can be used as generators. Thus, a RAG system with administrative documents as knowledge base is needed. As these documents are in part structured in tables, document preprocessing is a crucial step.

There are several goals for this thesis. Firstly, a RAG pipeline is built. Secondly, the best set-up for preprocessing the structured documents is determined empirically. Thirdly, the benefits of query expansion are investigated. Fourthly, possible hyperparameters are evaluated for the chunking algorithms and the best chunking method is

determined for the context at hand.

To rigorously benchmark the pipeline, a custom evaluation dataset on the three administrative documents was constructed. It comprises 73 question-answer pairs to evaluate answer correctness and an additional 14 unanswerable queries to test the pipeline’s negative rejection capabilities. The underlying codebase was intentionally designed to be highly modular, ensuring that future integration of new components or documents for departmental use is easily feasible.

The empirical evaluation of these goals yielded several key findings. First, combining preprocessing with chunking substantially elevated answer correctness from 0.423 without both preprocessing and chunking to over 0.570 on optimized configurations. For document conversion, direct LLM prompting significantly outperformed dedicated frameworks like Docling or existing literature strategies both for conversion to Markdown and for conversion to plain text.

In the end, the best chunkers used plain text documents, but the best chunkers using Markdown documents were not considerably behind. A key distinction here is that for plain text, chunks should be smaller, while for Markdown, table integrity is more important. Regarding chunking, a crucial insight emerged: more complex frameworks (such as LumberChunker or MiniRAG) failed on this specific task, and even successful semantic methods like Max-Min chunking required significant hyperparameter adaptation. This demonstrates that complex RAG methods cannot be applied out-of-the-box to small, homogeneous, administrative corpora. Instead, deterministic or highly tailored chunkers are more suitable. Surprisingly, Character Chunking, which was originally implemented as baseline, achieved the highest answer correctness overall. However, its ability for negative rejection was poor.

Ultimately, the most effective pipeline configuration for both answer correctness and negative rejection rate identified relies on LLM-based linearization, Max-Min semantic chunking, and a conservative cross-encoder threshold of 0.05, offering an optimal setup for resource-constrained environments.

The code for this thesis is publicly available at github.com/TrottingMouse/thesis-cl-rag-chatbot.

2 Data

2.1 Documents

The RAG-chatbot is based on three administrative documents from the Department of Computational Linguistics at the University of Düsseldorf: Prüfungsordnung (PO), Modulhandbuch (MH) and fachspezifische Prüfungsordnung (fPO)¹. PO requires minimal preprocessing, as it contains all the general information about studying and exams and is structured in paragraphs. MH has to be preprocessed, as all the modules for Computational Linguistics are shown in table-like structures with two columns. For fPO, preprocessing is strictly required as there is a table for every semester containing subjects and their credit points, time required, exams and administrative numbers.

¹The abbreviations are chosen differently in the code (Prüfungsordnung was only added later on). In the code, Prüfungsordnung is POges and fachspezifische Prüfungsordnung is PO.

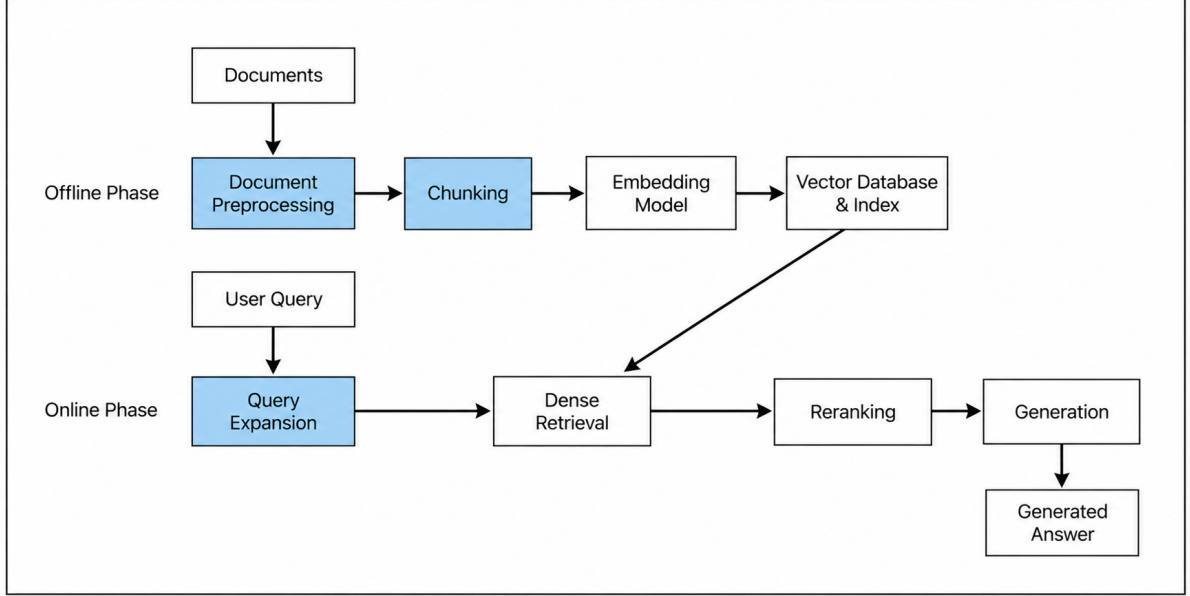


Figure 1: The RAG pipeline

2.2 Evaluation Dataset

To benchmark the performance of the RAG pipeline, a domain-specific evaluation dataset comprising 73 question-answer pairs was constructed. The dataset was created in collaboration with a student representative from the Department of Computational Linguistics. Candidate questions were generated through a combination of self-written queries and hand-selected suggestions produced by Google Gemini, which were subsequently refined to mirror real-world student inquiries. The answers were generated using Google Gemini, and manually checked and corrected. Table 1 provides representative sample pairs illustrating the different topic areas and difficulty profiles present in the evaluation dataset.

The composition of the dataset reflects strategic advice from the student representative, prioritizing topics that represent frequent points of friction throughout the academic life cycle. Consequently, a strong emphasis was placed on questions typical of the early stages of study (Sample pairs 1, 3, 5) and questions about when to take specific courses (Sample pair 1), exams in specific modules (Sample pair 3), and the mandatory internship (Sample pair 4). These topics were combined with broader administrative queries regarding studying in general (Sample pairs 2, 5) and specific questions concerning individual module contents.

To ensure a comprehensive benchmark, the dataset intentionally balances questions across varying levels of retrieval and synthesis difficulty. At the foundational level, it includes easier, direct queries where the answer can be read in one specific location (Sample pairs 2, 4, 5). The dataset also features moderately difficult questions where the target information is located in a single spot but remains hard to find and extract due to dense phrasing or complex tabular structures, particularly information from fPO (Sample pair 3). Finally, the benchmark includes complex queries that require the pipeline to retrieve multiple passages from different sections of a document, interpret them correctly, and synthesize them into a coherent answer, such as assembling a complete first-semester schedule (Sample pair 1).

The questions were consciously formulated in a short and colloquial manner in order

to simulate realistic user interactions. While colloquial phrasing occasionally introduces lexical ambiguity, questions were formulated to ensure unambiguous grounding against the underlying document corpus.

Table 1: Representative Sample Question-Answer Pairs from the Evaluation Dataset

#	User Question	Gold Answer
1	Welche Kurse sollte ich im ersten Semester belegen?	Laut dem exemplarischen Studienverlaufsplan solltest du im ersten Semester folgende Kurse belegen: Mathematische Grundlagen, Einführung in die computationelle Logik, Einführung in die Computerlinguistik, Einführung in die Programmierung mit Python, Morphologie und Syntax.
2	Bis wann kann ich mich von einer Klausur ohne Angabe von Gründen abmelden?	Von einer Modulprüfung in Form einer Klausur oder einer mündlichen Prüfung kann sich die Kandidatin oder der Kandidat bis spätestens eine Woche vor dem jeweiligen Prüfungstermin abmelden.
3	Gibt es eine Klausur in Wahrscheinlichkeitstheorie?	Nein, im “Basismodul – Statistische Methoden in der Computerlinguistik”, zu dem die Wahrscheinlichkeitstheorie gehört, gibt es gar keine Modulprüfung und somit auch keine Klausur. Es müssen lediglich Studienleistungen erbracht werden.
4	Wann soll ich das Praktikum machen?	Es wird empfohlen, das Berufsfeldpraktikum nach dem vierten oder fünften Semester zu absolvieren.
5	Gibt es eine generelle Anwesenheitspflicht für Vorlesungen?	Nein, bei Vorlesungen kann keine Anwesenheitspflicht festgelegt werden.

In order to measure how well the pipeline copes with questions that cannot be answered from the documents, another dataset with 14 questions unanswerable from the corpus was created (see Table 2 for examples). These questions span from questions related to administrative topics of Computational Linguistics (Sample pair 1) over subject-related questions (Sample pair 2) and university-related questions (Sample pair 3) to questions completely out of context (Sample pair 4).

3 RAG pipeline

3.1 Code Architecture

To facilitate systematic experimentation and ensure future extensibility, the codebase of the RAG pipeline is built upon a strictly modular architecture. A fundamental architectural choice is the strict separation of the pipeline into offline and online execution

Table 2: Sample Question-Answer Pairs from the Negative Rejection Dataset

#	User Question	Gold Answer
1	Welche Fristen gelten für die Prüfungsanmeldung der Klausur in Morphologie und Syntax?	Dazu enthalten die bereitgestellten Dokumente keine Informationen.
2	In welchem Jahr wurde die Chomsky-Hierarchie zum ersten Mal publiziert?	Dazu enthalten die bereitgestellten Dokumente keine Informationen.
3	Wie hoch ist der aktuelle Semesterbeitrag an der Heinrich-Heine-Universität Düsseldorf?	Dazu enthalten die bereitgestellten Dokumente keine Informationen.
4	Wie viele Kilometer ist die Sonne durchschnittlich von der Erde entfernt?	Dazu enthalten die bereitgestellten Dokumente keine Informationen.

phases. The offline phase handles the computationally expensive, one-time tasks of document preprocessing, chunking, and vector indexing. The online phase manages real-time interactions, encompassing query expansion, retrieval, reranking, and generation.

Each distinct step of the pipeline is encapsulated within its own module using an object-oriented design approach. The architecture relies on abstract base classes for every pipeline component (e.g., preprocessors, chunkers, retrievers, and generators). Specific algorithms and models are implemented as subclasses that inherit from these base classes. This design pattern ensures that replacing an existing component or integrating a new one – such as introducing a hybrid retriever or a novel chunking algorithm – only requires writing a new implementation of the respective base class, leaving the overarching pipeline logic completely untouched.

To manage the high number of possible pipeline configurations, component selection and hyperparameter choices are handled entirely via a central configuration file. A dedicated Python factory script reads this configuration file and dynamically builds the requested pipeline at runtime by instantiating the specified classes. This allows for rapid component switching and modifications of hyperparameters without the need to modify any source code.

Finally, the architecture is designed for easy maintenance of the knowledge base. New PDF files can be seamlessly integrated into the RAG system by simply placing the raw files into a designated documents directory and registering the new document in the configuration file. For usage of the GeminiMarkdownProcessor, a Markdown file in txt format converted by an LLM needs to be placed in the cached documents folder. The factory and offline modules will then automatically process, chunk, and index the new data during the next offline run.

3.2 Document preprocessing

As two of the documents contain tables, a direct conversion to raw, unformatted text is not sensible. Min et al. (2024) identified four methods to handle tables for RAG. First, the table can be converted to Markdown. Second, it can be converted to simple sentences using automatic templates. Third, traditional (non-generative) language models can be

fine-tuned for this task. Fourth, LLMs can be used to convert the tables to linearized text. In the paper’s comparison of the four strategies, LLM-based conversion performed best, followed by Markdown, which still gave good generation results. Thus, for the RAG pipeline, both Markdown and LLM-based conversion are used. For conversion to Markdown, Docling (Deep Search Team 2024) and conversion by an LLM are tested. Docling is a framework based on pre-trained lightweight vision models and a deep learning model specialized on table conversion. The prompt for the conversion via LLM can be found in Listing 1.

Listing 1: Prompt for conversion to Markdown

```
Convert this document to markdown. Pay attention to table
structure and the separation of tables and modules.
```

For conversion to text by an LLM, the technique from Min et al. and a self-written prompt are compared. In the paper, the table is first converted into a nested bracket structure and then converted to text by the LLM. The conversion prompt from the paper is translated into German. The self-written prompt (see Listing 2) takes the Markdown documents and converts them directly to text.²

Listing 2: Prompt for Linearization from Markdown to Text

```
Wandle das folgende Markdown-Dokument in klaren,
informationsdichten Fließtext um, der für eine semantische
Suchmaschine (RAG-Pipeline) optimiert ist.
```

Befolge dabei strikt diese Regeln:

1. Informationserhalt: Übernimm ausnahmslos jede Information, jede Zahl und jedes Detail.
2. Hohe Entitätsdichte (WICHTIG): Vermeide Pronomen (er, sie, es, diese) über Absatzgrenzen hinweg. Wiederhole stattdessen immer wieder die konkreten Eigennamen, Produktnamen oder Fachbegriffe, damit jeder Absatz auch isoliert verständlich bleibt.
3. Tabellen & Listen: Wandle Tabellen und Aufzählungen in zusammenhängende Textabsätze um. Formuliere die Beziehungen zwischen Spalten/Zeilen in ganzen Sätzen aus.
4. Überschriften als Kontext: Nutze Überschriften, um den darauffolgenden Absatz mit einem klaren Kontext-Satz zu beginnen (z. B. statt nur "Spezifikationen:" -> "Die technischen Spezifikationen des Geräts X umfassen...").
5. Formatierung: Entferne alle Markdown-Zeichen (*, #, -, |). Behalte klare Absatzumbrüche bei, um logische Trennungen beizubehalten.

Input Text:

```
---
{text}
---
```

²Parts of this section have been taken from the previous literature review

3.3 Chunking

With early transformer models and the first RAG systems, chunking was only done in a fixed size of tokens, words or characters (Dai et al. 2019; Lewis et al. 2020). For this thesis, Fixed Character Chunking was implemented as a baseline chunker. With Dynamic Token Chunking and Fixed Paragraph Chunking, two other simple deterministic Chunkers were implemented. Dynamic Token Chunking sets a maximum number of tokens and cuts off at the last sentence ending before the limit. Fixed Paragraph Chunking packs a fixed number of paragraphs into one chunk, while Fixed Character Chunking does the equivalent with a set number of characters. Fixed Character Chunking and Fixed Paragraph Chunking were used in combination with both preprocessing techniques, while Dynamic Token Chunking was only used with linearized text, as it needs sentences for splitting and does not work with Markdown tables.

A big problem of chunking in general, and especially fixed-size chunking, is the splitting of contexts (context problem). A word or a sentence in one chunk can reference parts of the previous chunk or even a more distant one. This information gets lost in the chunking process. The first solution to this problem, which was derived from the training of BERT (Devlin et al. 2019), was overlapping chunks. Overlap is a certain number of tokens or characters that two adjacent chunks share, giving the beginning and the end of a chunk more context. For Fixed Character Chunking, Fixed Paragraph Chunking, and Dynamic Token Chunking, an overlap was tested in the experiments.

Another solution to the context problem is semantic chunking. This method embeds every sentence and separates two adjacent sentences into different chunks if their similarity is below a certain threshold (Kamradt 2024). However, later studies found that semantic chunking had only a slight, non-significant edge over fixed-size chunking for large contexts, which does not justify the computational overhead (Qu, Tu, and Bao 2025; Bannani and Moslonka 2026). This led to the development of Max-Min-Chunking, an improved semantic chunker achieving significantly better retrieval and RAG accuracy than both previous semantic chunkers and simple fixed-size chunkers (Kiss, Nagy, and Szilágyi 2025). The algorithm uses "dynamic clustering". With Max-Min semantic chunking, a sentence is allocated to the same chunk as the previous one if it shares a higher similarity with any sentence in the current chunk than the pair with the lowest similarity in the chunk. This dynamic threshold can even be dampened or increased by a factor c . It is also getting higher with increasing chunk size through a sigmoid-based function. There is, however, a hard baseline for cosine similarity (`fixed_threshold`). The Max-Min chunker was tested for linearized text in the chunking experiments.

An alternative approach to deal with the context problem is improving the workflow of chunking. Contextual Retrieval addresses it by prepending contextual text to every chunk, situating it inside the bigger document. This significantly reduces the retrieval failure rate (Anthropic 2024). With Late Chunking, JinaAI proposed a more profound change to the retrieval pipeline. Late Chunking embeds all tokens first so the word embeddings have the context of the whole text. Only then are the documents chunked using any chunking algorithm. In the end, mean pooling is done on the vectors in each chunk (Günther et al. 2024). However, both methods are more sensible for in-corpus retrieval, meaning retrieval from a big, thematically heterogeneous corpus. For this thesis, there are only three, thematically homogeneous documents included, making the task very similar to in-document retrieval (retrieval from one long, thematically homogeneous document). Here, contextualized chunking with standard methods is not advised (Zhou

et al. 2026). Instead, two structure-based methods enriching table chunks with context were implemented specifically for the documents.

The first one, Whole Table Paragraph Chunker, splits the structured Markdown documents into entire tables (for fPO) or modules (for MH). Each table chunk receives the abbreviation legend and its heading as contextualization. The normal, unstructured text in the Markdown documents is split into paragraphs. The second one, the Split Table Paragraph Chunker, splits semester tables at module boundaries. It also divides module descriptions by individual field lines. This makes retrieval more precise and reduces the context for the LLM. Because context gets lost here, the relevant semester or module headings are prepended to each chunk. Additionally, abbreviations are replaced by their full words, and sub-course letter references are resolved to their actual names. Normal text, like in PO, is split into sentences to stay true to the small chunk size. Both Table chunkers are fitted to the documents and might need to be adjusted for new documents.

With transformer-based LLMs getting more reliable, the models were used for more mechanical tasks like chunking. LumberChunker successfully implemented this approach (Duarte et al. 2024). The document is split into paragraphs. Iteratively, all paragraphs up to a certain token length are fed to an LLM to decide where the content changes, which is where the current chunk is cut. The resulting chunks align closely with manually created chunks and thus perform quite well. For in-document retrieval, LumberChunker performed particularly well in a comparative study (Zhou et al. 2026). Thus, LumberChunker was selected as one of the chunkers for the chunking experiments for linearized text. The prompt from the original LumberChunker paper was translated into German (Listing 3).³

Listing 3: LumberChunker translated Prompt

```
Du wirst als Eingabe ein deutsches Dokument mit
Paragraphen erhalten, die durch 'ID XXXX: <text>'
identifiziert werden.

Aufgabe: Finde den ersten Paragraphen (nicht der erste),
bei dem sich der Inhalt im Vergleich zu den vorherigen
Paragraphen deutlich ändert.

Ausgabe: Gib die ID des Paragraphen mit dem
Inhaltswechsel im Format zurück: 'Answer: ID XXXX'.

WICHTIGE REGELN:
1. Erkläre NICHT deine Gedankengänge.
2. Gib NUR die ID des Paragraphen mit dem Inhaltswechsel
im Format zurück: 'Answer: ID XXXX'.
3. Vermeide sehr lange Gruppen von Paragraphen. Strebe
ein gutes Gleichgewicht zwischen der Identifizierung
von Inhaltswechseln und der Beibehaltung handhabbarer
Gruppen an.

Dokument:
{window_text}
```

³Parts of this section have been taken from the previous literature review

3.4 Indexing and Retrieval

After chunking, the chunks are embedded using a sentence transformer model. The model used is jinaai/jina-embeddings-v5-text-nano, a cutting-edge embedding model with only 239M parameters, which is also well-suited for German language according to the MTEB leaderboard (Akram et al. 2026; Muennighoff et al. 2022; Enevoldsen et al. 2025). A particularly small model was chosen in order to keep resource usage limited, also for indexing and retrieval. FAISS was used for vector storage (Douze et al. 2024).

3.5 Query expansion

Retrievers suffer from an elementary flaw: the query is structurally very different from the document passages. For many models, this is solved during training, so a query is embedded differently than a document (Akram et al. 2026). Even this does not always solve the discrepancy. Query expansion techniques aim to bridge the structural gap by rewriting the query and retrieving against the expanded queries. Two of those techniques were tested and compared to the baseline with no query expansion. The SLM used for generating the RAG answer was also used for query expansion.

HyDE (Hypothetical Document Embeddings) is one of the first and most used query expansion techniques (Gao et al. 2023). It uses an LLM to generate a hypothetical answer to the query without using any documents. Afterwards, it retrieves against this generated answer. The prompt used is taken from the original paper translated to German (Listing 4).

Listing 4: Prompt for HyDE Query Expansion

```
Bitte schreibe einen Textabschnitt, um die folgende Frage  
zu beantworten: {query}
```

Jagerman et al. (2023) compared different query expansion techniques as zero-shot prompts (without examples in the prompt) and few-shot prompts (with some examples in the prompt). The main metric used in the original paper was recall@1K, which is the proportion of all actual relevant items that the system successfully returns in its top 1,000 retrieved results. The prompt with the highest recall was zero-shot Chain-of-Thought, a prompt that explicitly asks the LLM to give the rationale before answering. This is the second prompt tested for query expansion (Listing 5).

Listing 5: Prompt for CoT Query Expansion

```
Beantworte die folgende Anfrage:  
{query}  
Gib die Begründung an, bevor du antwortest.
```

3.6 Reranking

Reranking improves retrieval by retrieving k documents, reranking the retrieved results and only returning the top- n documents (Nogueira and Cho 2019), thus $k > n$. The reranker is more computationally expensive than the retriever, as it uses cross-encoding instead of a simple vector similarity computation. Cross-encoding means processing the query and a retrieved document jointly through a transformer model. Afterwards, a classification token is returned to indicate the similarity between the query and the document. The reranker used here is jinaai/jina-reranker-v3 (Wang, Li, and Xiao 2025).

3.7 Generation

Due to practical requirements for later deployment, a very small language model was chosen for generation. The model is Qwen/Qwen3.5-2B. This model was chosen due to its popularity and its good performance on multilingual benchmarks compared to its size (Qwen 2026). Some early tests were done with Qwen/Qwen3.5-0.8B, but the output in German was too unreliable.

The RAG prompt was kept simple and straightforward. To limit response length and time, a 256 token cut-off was configured for the generator. In first tests, the model added a lot of unnecessary information to its answer, so the answer was often cut off and the relevant facts for answering the query were omitted. Thus, a phrasing to prevent this was included (see Listing 6). As a consequence, the relevant facts were in the answers more often and less irrelevant facts were given.

Listing 6: Prompt for the generative model

```
Du bist ein hilfreicher Assistent. Beantworte die Frage
basierend auf dem gegebenen Kontext. Halte dich kurz
und gib nur die Informationen, nach denen gefragt
wurde. Wenn die Antwort nicht im Kontext zu finden ist
, antworte: 'Dazu enthalten die bereitgestellten
Dokumente keine Informationen'.
Frage: {query}
Kontext:
{context_str}
```

3.8 Left-out Methods

Several RAG frameworks created by previous papers were tested or considered but in the end not included for various reasons.

TableRAG (S.-A. Chen et al. 2024) uses an LLM to create queries for pandas. However, this does not take mixed documents into consideration, where there are both text and tables. Another TableRAG framework based on SQL (Yu, Jian, and Chen 2025) takes both into account. However, the integration for the experiments and especially for later deployment was too complex due to database connections and a complex architecture.

TabRAG (Si et al. 2025) separates and includes both text and tables into RAG using a vision learning model. This was tested on the faculty GPU server, but the model turned out to be too large for the resources available on the server.

MiniRAG (Fan et al. 2025) is a graph-based framework specifically designed for SLMs. It extracts entities and relationships from the documents and links entities to context. With the university documents, this method was hardly functionable. Some entities could be extracted, but the algorithm detected no relationships between them. Thus, the elaborate retrieval algorithm did not work and the generation results were worse than even for the baseline without chunking.

4 Experiments

4.1 Evaluation Setup

Due to the large variety in preprocessing and chunking techniques, the retrieved chunks of different experiments are very different in structure and content. Thus, classic retrieval evaluation techniques like determining precision and recall from token spans fail here or would have come with a huge manual overhead. Instead, an LLM was used to evaluate retrieval and generation results (LLM-as-a-judge). Previous studies using GPT-4 have shown that LLMs achieve an agreement rate on par with the inter-annotator agreement for this task (Y. Liu et al. 2023; Zheng et al. 2023). In order to counter the problem of LLMs favoring their own answers (W.-L. Chen et al. 2025), GPT 4o-mini was used for evaluation, as most of the preprocessed chunks were preprocessed by Google Gemini.

The metrics in use stem from RAGAS (Es et al. 2024). The original RAGAS paper introduced LLM-as-a-judge for RAG specifically with three metrics. Faithfulness assesses whether the answer can be inferred from the context. This is done by splitting the answer into statements and dividing the number of grounded statements by the total number of statements. Answer relevancy assesses whether the answer addresses the query. This metric did not give consistent results on preliminary tests and was therefore not used here. Context Relevance judges if the context only contains relevant information, but is dependent on a sentence-based structure of the chunks and thus also not used here.

Instead, two other metrics from the RAGAS package were used. The first is context recall. This metric measures how many of the relevant chunks are successfully retrieved. It divides the number of claims in the reference supported by retrieved chunks by the total number of claims in the reference. The second is answer correctness. This measures the correctness of the final generated answer by combining a factual F1 score with embedding similarity. The F1 score is calculated as follows:

$$\text{F1 Score} = \frac{|\text{TP}|}{(|\text{TP}| + 0.5 \times (|\text{FP}| + |\text{FN}|))}$$

TP (true positives) are facts extracted from the generated answer that are present in the reference answer. FP (false positives) are facts present in the generated answer that are not present in the reference answer. FN (false negatives) are facts present in the reference answer that are not present in the generated answer. Answer correctness was the main metric for evaluating the pipeline because the goal of this thesis was to find a pipeline configuration that worked well in combination with the SLM, and answer correctness tests the overall result. However, the other metrics helped to identify problems with certain pipeline configurations.

An important attribute of RAG pipelines which could not be captured by the previous three metrics is negative rejection. This is the ability to reject questions which cannot be answered with the given context. Negative rejection is important, as a hallucinated answer for an administrative question could have severe consequences for students. For the RAGAS framework, a metric with a custom prompt was implemented to measure negative rejection (see Listing 7). This metric was used for the second dataset containing unanswerable queries.

Listing 7: Negative Rejection Prompt

```
Did the model reject the query as not answerable from the
given context?
```

4.2 Pipeline Configuration

The RAG pipeline encompasses numerous configurable components and hyperparameters. Because evaluation using LLM-as-a-judge is computationally and resource-intensive, exhaustive grid searches across the entire parameter space were impractical. To address this, four preliminary sequential experiments were conducted to independently optimize document preprocessing, chunking hyperparameters, context size, and query expansion prior to the main evaluation. In each stage, the optimal parameters identified in preceding steps were adopted. For instance, following the preprocessing benchmark, only the top-performing preprocessors were evaluated in subsequent pipeline stages. To conserve computational resources during parameter tuning, these preliminary experiments were executed on a representative, thematically balanced subset of 15 question-answer pairs drawn from the primary evaluation dataset.

4.3 Document Preprocessing Experiment

In the first experiment, document preprocessing strategies were evaluated in two stages. First, Markdown conversion produced by Gemini was compared against output from Docling (Deep Search Team 2024). Second, the superior Markdown preprocessor was combined with the custom prompt for text linearization and evaluated against the multi-step table linearization method from Min et al. (2024). The top-performing approach from each comparison was selected for all subsequent pipeline configurations. To prevent biasing the evaluation toward table-specific chunking strategies, the Fixed Paragraph Chunker (size 1, overlap 0) was employed as a baseline due to its general applicability to document structures. Because individual paragraphs could be relatively long (up to 600 tokens), the retrieval parameters were fixed at $k = 9$ retrieved documents (*top-k*) and $n = 3$ reranked documents (*top-n*).

4.4 Grid Search for Chunking Hyperparameters

For the deterministic strategies (Fixed Character, Fixed Paragraph, and Dynamic Token Chunking), chunk size and overlap parameters were systematically tuned such that individual chunks remained below 1500 tokens in most cases. Fixed Character and Fixed Paragraph strategies were tuned independently for Markdown and linearized documents to accommodate structural differences. For Fixed Paragraph Chunking, an overlap of 1 paragraph was evaluated for chunk size 2. For Fixed Character and Dynamic Token Chunking, a 10% overlap was compared against a zero-overlap baseline.

For Max-Min Chunking, the hard cosine threshold (*hard_threshold*) and dampening parameter (c) were optimized via grid search. Initial exploratory tests revealed that the baseline hyperparameter settings proposed by Kiss, Nagy, and Szilágyi (2025) caused infinite aggregation loops resulting in single, oversized chunks. When similarities near 0.6 occurred early in a chunk while the sigmoid-based dynamic threshold remained low, multiple moderately dissimilar sentences were aggregated together. This artificially expanded the semantic diversity of the chunk, which in turn increased the maximum similarity score for subsequent candidate sentences and further lowered the dynamic threshold. This behavior was largely driven by the high thematic homogeneity of the university administrative documents. To prevent this runaway effect, candidate values for c and *hard_threshold* were restricted to strictly higher ranges than those reported in the original paper.

LumberChunker has no tunable hyperparameters which would significantly change the chunking process, so this strategy was omitted in the grid search.

4.5 Reranking Threshold Experiment

Determining the optimal number of retrieved (top_k) and reranked (top_n) chunks presents a structural trade-off. While maintaining fixed values for top_k and top_n across all configurations is straightforward, it leads to uneven context lengths during generation due to large variations in average chunk size across different chunking strategies. To establish fair comparisons and respect context limitations, top_n and top_k were dynamically derived from the average chunk size (S_{avg}) such that context size remained on average under 1500 tokens:

$$N_{top} = \max \left(1, \left\lfloor \frac{1500}{S_{avg}} \right\rfloor \right), \quad K_{top} = 3N_{top} \quad (1)$$

The 1500-token threshold was selected because context performance in open-weight models around 8B parameters degraded significantly between 4000 and 8000 input tokens (Leng et al. 2024). Setting a conservative budget for the 2B generator model minimizes lost-in-the-middle phenomena.

However, a strict token cap may disadvantage fine-grained chunking strategies designed to provide short, highly targeted context. To evaluate whether variable context lengths improve performance, a similarity thresholding mechanism was tested. Under this setup, fewer chunks were passed to the generator if the cross-encoder similarity scores of lower-ranked chunks fell below a specific cutoff. This mechanism was evaluated for all preprocessing/chunking combinations.

4.6 Query Expansion Experiment

The two query expansion techniques – Hypothetical Document Embeddings (HyDE) and Chain-of-Thought (CoT) prompting – were evaluated against a baseline without query expansion. To maintain low resource requirements, the generator SLM (Qwen3.5-2B) was also used to generate the expanded queries. All three query expansion methods were also tested for all preprocessing/chunking setups.

4.7 Main Experiment: Preprocessor and Chunker Comparison

In the final experiment, all viable combinations of preprocessors and chunkers – configured with the optimal hyperparameters established in the preceding steps – were evaluated on the complete dataset of 73 question-answer pairs as well as the 14-item negative rejection dataset. Combinations that were structurally incompatible (such as applying table-aware chunkers to linearized text) were excluded from the main comparative benchmark. Two experiments were run as baselines for comparison. The first one used no preprocessing and no chunking. Thus, the SLM was given all three documents as raw text without any formatting. For the second baseline experiment, the documents were preprocessed to linearized text by the optimized preprocessor setup, but were not chunked. These baseline experiments served to verify that the introduction of chunking and RAG had a measurable effect on generation results and that the lost-in-the-middle phenomenon was still relevant for an SLM, even with only three documents.

5 Results

5.1 Preprocessing choices

GeminiMarkdownProcessor outperforms DoclingMarkdownProcessor for Markdown conversion. Together with DirectLLMProcessor, it also outperforms PaperLLMProcessor for text linearization (see Table 3).

Table 3: Comparison of Document Preprocessors

Preprocessor	Answer Accuracy
DoclingMarkdownProcessor	0.454
GeminiMarkdownProcessor (GM)	0.485
GeminiMarkdownProcessor+DirectLLMProcessor (LLM)	0.531
PaperLLMProcessor	0.439

5.2 Chunking Parameters

The results for the hyperparameter grid search across chunking strategies and preprocessors are presented in Table 4. For Fixed Character Chunking on Markdown documents, larger chunk sizes without overlap yielded the highest performance, peaking for a chunk size of $S = 1500$ characters. Introducing an overlap mostly degraded performance for Markdown, most noticeably at $S = 500$. For Fixed Paragraph Chunking on Markdown, a chunk size of $S = 2$ paragraphs without overlap achieved the highest score, outperforming both smaller ($S = 1$) and larger ($S = 3$) paragraph windows as well as configurations with paragraph overlap ($S = 2, O = 1$).

When applied to linearized text, Fixed Paragraph Chunking performed best at a smaller size of $S = 1$ paragraph, with performance declining as the paragraph count or overlap increased. For Fixed Character Chunking on linearized text, a smaller chunk size with overlap ($S = 500, O = 50$) achieved the best result. For Dynamic Token Chunking, a smaller context window with a 10% overlap ($S = 150, O = 15$) proved optimal. Finally, for Max-Min Chunking, tuning the hard threshold (τ) and dampening parameter (c), $\tau = 0.75$ and $c = 1.3$ achieved the best results for this strategy. Both values stand in the middle of the ranges for their respective parameter and thus confirm the testing range.

Table 4: Grid Search Results for Chunking Hyperparameters

Prep.	Chunker	Params	Acc.	Prep.	Chunker	Params	Acc.
GM	FixedChar	$S=500, O=0$	0.6612	GM+LLM	MaxMin	$\tau=0.70, c=1.3$	0.4982
GM	FixedChar	$S=500, O=50$	0.4191	GM+LLM	MaxMin	$\tau=0.70, c=1.4$	0.5204
GM	FixedChar	$S=1000, O=0$	0.5598	GM+LLM	MaxMin	$\tau=0.70, c=1.5$	0.5247
GM	FixedChar	$S=1000, O=100$	0.5967	GM+LLM	MaxMin	$\tau=0.75, c=1.2$	0.5720
GM	FixedChar	$S=1500, O=0$	0.6811	GM+LLM	MaxMin	$\tau=0.75, c=1.3$	0.5861
GM	FixedChar	$S=1500, O=150$	0.5887	GM+LLM	MaxMin	$\tau=0.75, c=1.4$	0.5440
GM	FixedPara	$S=1, O=0$	0.6166	GM+LLM	MaxMin	$\tau=0.75, c=1.5$	0.5819
GM	FixedPara	$S=2, O=0$	0.6274	GM+LLM	MaxMin	$\tau=0.80, c=1.2$	0.5392
GM	FixedPara	$S=2, O=1$	0.5939	GM+LLM	MaxMin	$\tau=0.80, c=1.3$	0.5524
GM	FixedPara	$S=3, O=0$	0.5702	GM+LLM	MaxMin	$\tau=0.80, c=1.4$	0.5496
GM+LLM	FixedPara	$S=1, O=0$	0.6605	GM+LLM	DynToken	$S=150, O=0$	0.6058
GM+LLM	FixedPara	$S=2, O=0$	0.5988	GM+LLM	DynToken	$S=150, O=15$	0.6386
GM+LLM	FixedPara	$S=2, O=1$	0.5421	GM+LLM	DynToken	$S=250, O=0$	0.6084
GM+LLM	FixedPara	$S=3, O=0$	0.5275	GM+LLM	DynToken	$S=250, O=25$	0.5738
GM+LLM	FixedChar	$S=500, O=0$	0.6019	GM+LLM	DynToken	$S=500, O=0$	0.5975
GM+LLM	FixedChar	$S=500, O=50$	0.6184	GM+LLM	DynToken	$S=500, O=50$	0.4894
GM+LLM	FixedChar	$S=1000, O=0$	0.5313				
GM+LLM	FixedChar	$S=1000, O=100$	0.5716				
GM+LLM	FixedChar	$S=1500, O=0$	0.5619				
GM+LLM	FixedChar	$S=1500, O=150$	0.5261				

Note: GM = GeminiMarkdownProcessor, LLM = DirectLLMProcessor, S = Size, O = Overlap, τ = Threshold.

5.3 Context Size and Reranking Threshold

The performance evaluation across reranking thresholds is presented in Table 5. Overall, applying a similarity threshold yielded mixed effects depending on the chunking strategy and preprocessing format. For several configurations, introducing a moderate threshold of 0.05 improved answer correctness compared to the baseline without thresholding (threshold = 0.00). Specifically, Whole Table Chunking peaked at 0.647, Dynamic Token Chunking reached 0.639, Max-Min Chunking improved to 0.584, and the LLM Chunker increased to 0.526 under the 0.05 cutoff.

Conversely, for standard Paragraph and Character chunkers across both Markdown and Direct preprocessing, introducing a threshold generally led to a decline in answer correctness compared to the 0.00 baseline. An exception to this pattern was observed with the Split Table Chunker, which performed poorest at the 0.05 threshold (0.549) but achieved its highest accuracy at a stricter threshold of 0.10 (0.591). Across most other strategies, however, raising the threshold to 0.10 resulted in noticeable performance drops, most prominently seen in Whole Table Chunking (falling to 0.544) and Dynamic Token Chunking (falling to 0.518).

Table 5: Comparison of Reranking Thresholds for each Preprocessor/Chunker Combination with Average Tokens per Chunk

Preprocessing	Chunker	Avg. Tokens	Reranking Threshold		
			0.00	0.05	0.10
Markdown (GM)	Paragraph	221.9	0.627	0.593	0.601
	Character	385.2	0.649	0.584	0.580
	Whole Table	97.4	0.645	0.647	0.544
	Split Table	40.4	0.579	0.549	0.591
Linearized (GM+LLM)	Paragraph	375.6	0.598	0.592	0.524
	Character	121.6	0.600	0.568	0.579
	Dynamic	217.5	0.630	0.639	0.518
	LLM Chunker	752.2	0.504	0.526	0.516
	Max-Min	77.2	0.576	0.584	0.578

5.4 Query Expansion

There was no clear best strategy for query expansion across all preprocessing and chunking strategies (Table 6). However, some trends could be observed. Overall, Chain-of-Thought (CoT) proved beneficial in more cases than Hypothetical Document Embeddings (HyDE), achieving the highest answer correctness in four out of the nine evaluated preprocessor-chunker pairings compared to only one for HyDE. CoT achieved notable improvements on Direct LLM preprocessed documents when paired with Character Chunking, Paragraph Chunking and Max-Min-Chunking as well as on Markdown documents with Whole Table Chunking. However, the baseline of no query expansion also achieved good results, being the optimal strategies in four cases.

Furthermore, query expansion exhibited a higher overall impact when applied to Markdown preprocessed documents than to linearized text. For Markdown documents, query transformation led to substantial variations in performance: HyDE yielded a performance gain for Paragraph Chunking, whereas CoT caused a marked drop. Conversely, for linearized text, applying query expansion had a much more subtle effect. Even in cases where the baseline without processing achieved the top score—such as with Dynamic Token Chunking, LLM Chunking, and Max-Min Chunking, the margin between the baseline and the expansion techniques remained very narrow.

Table 6: Evaluation of RAG Pipeline Answer Correctness Across Query Processors.

Preprocessing	Chunker	Query Processor		
		No Processing	HyDE	CoT
Markdown (GM)	Paragraph	0.556	0.589	0.501
	Character	0.689	0.660	0.603
	Whole Table	0.620	0.640	0.647
	Split Table	0.551	0.514	0.459
Linearized (GM+LLM)	Paragraph	0.639	0.588	0.646
	Character	0.612	0.579	0.677
	Dynamic	0.636	0.624	0.629
	LLM Chunker	0.507	0.502	0.489
	Max-Min	0.585	0.585	0.595

5.5 Main Experiment

The main experiment demonstrates the clear advantage of employing proper preprocessing and chunking strategies. As shown in Table 7, the baseline without any preprocessing or chunking achieved an answer correctness of only 0.423. Introducing preprocessing to create linearized text without chunking improved the results solidly to an answer correctness of 0.485. However, combining preprocessing with chunking yielded the best outcomes, with all tested combinations achieving an answer correctness between 0.51 and 0.58, outperforming both baselines.

Within the Markdown preprocessing configurations, the Whole Table and Paragraph chunkers demonstrated very similar and strong results across all metrics. Both achieved good answer correctness and high context recall, with the Whole Table strategy reaching the highest context recall (0.902) among all nine evaluated strategies. Conversely, the Split Table and Character chunking methods performed similarly poorly on Markdown documents, yielding both low answer correctness and low context recall.

For linearized text preprocessing, Character and Max-Min Chunking emerged as the best-performing chunkers overall. Character chunking on linearized text achieved the highest answer correctness (0.577) and the highest faithfulness (0.740), alongside a slightly above-average context recall. Max-Min Chunking also performed very well, achieving a high faithfulness, though its context recall remained mid-range. In contrast, LLM chunking showed generally poor performance on linearized text, resulting in low answer correctness (0.517) and very low context recall. Furthermore, while the Paragraph chunker on linearized text achieved high context recall and faithfulness, this did not translate to strong final generation quality, resulting in an answer correctness of only 0.533.

Comparing the two preprocessing paradigms, faithfulness was generally higher for linearized text than for Markdown. With the exception of the underperforming LLM chunker, all linearized text strategies achieved higher faithfulness scores than all tested Markdown strategies.

Another notable observation is the behavior of the Dynamic Token chunker on linearized text. It achieved high context recall but relatively low faithfulness, leading to a mid-range answer correctness (0.544). However, it managed by far the highest negative rejection rate (0.929).

More broadly, the negative rejection rate did not show a strong correlation with answer correctness. For instance, Character chunking on Markdown had very low answer correctness but a very high negative rejection rate (0.857). Conversely, Character chunking on linearized text achieved the highest answer correctness but the lowest negative rejection rate (0.643). Among all configurations, Max-Min Chunking stood out as the only strategy that effectively balanced both high answer correctness and a strong negative rejection rate.

Table 7: Evaluation of RAG Pipeline Across Preprocessing and Chunking Strategies Across All Metrics.

Preprocessing	Chunker	CR	Faith.	AC	NRR
No Preprocessing	No Chunking			0.423	0.571
Markdown (GM)	Paragraph	0.856	0.683	0.553	0.786
	Character	0.775	0.672	0.515	0.857
	Whole Table	0.902	0.663	0.556	0.786
	Split Table	0.749	0.687	0.515	0.714
Linearized (GM+LLM)	No chunking			0.485	0.643
	Paragraph	0.862	0.737	0.533	0.786
	Character	0.839	0.740	0.577	0.643
	Dynamic	0.861	0.689	0.544	0.929
	LLM Chunker	0.745	0.671	0.517	0.857
	Max-Min	0.829	0.737	0.570	0.857

Note: CR = Context Recall, Faith. = Faithfulness, AC = Answer Correctness, NRR = Neg. Rejection Rate.

6 Discussion

6.1 Preprocessing methods

The first part of the preprocessing experiment suggests LLMs can outperform elaborate frameworks at complicated tasks. Docling was specifically designed for converting documents containing tables to Markdown. Still, Gemini performed better with a simple one line prompt.

The good results of the text linearization technique from Min et al. (2024) could not be reproduced. Instead, also here, a custom prompt using the Markdown documents as input achieved a better answer correctness. One explanation for this could be the direct translation of the table transformation prompt from the paper into German, which might not have been understood properly by the model. Another possible explanation is that the prompt was highly fitted to the datasets used in the paper. As the university documents look significantly different, the combination of pdf-to-text and prompt from the paper might be ill-fitted here.

6.2 Chunking Parameters

The grid search results reveal a clear divergence in optimal chunking strategies depending on the preprocessing format, highlighting the different ways Small Language Models (SLMs) process structured versus dense text.

For Markdown documents, applying zero overlap proved to be the most effective approach. Introducing overlap in Markdown duplicates unnecessary structural syntax (such as table delimiters and formatting tokens) across chunks. This not only wastes the limited context window but also feeds the SLM redundant structural noise that is difficult to process. Furthermore, Fixed Paragraph Chunking on Markdown performed best with a chunk size of two paragraphs. With a size of one, a table’s explanatory header or introductory paragraph is always separated from the table itself. A chunk size of two often prepends some necessary context to the table. For Fixed Character Chunking, a large chunk size was important to limit table fragmentation.

In contrast, when using linearized text, Fixed Paragraph Chunking peaked with a chunk size of just one paragraph. The custom prompt utilized by the `DirectLLMProcessor` explicitly enforces “high entity density” and explicit contextual relationships. Consequently, a single paragraph is already heavily saturated with information. Feeding multiple data-dense paragraphs to the 2B-parameter generator overloads its context processing capabilities, triggering the “lost-in-the-middle” phenomenon where the model fails to isolate the correct facts.

For deterministic, sub-paragraph chunkers operating on linearized text—namely Fixed Character and Dynamic Token chunking—smaller chunk sizes consistently yielded better results. Smaller sizes generate more specialized, targeted chunks, allowing the retriever to supply the SLM with highly specific facts. However, because these chunkers are fundamentally blind to natural semantic boundaries (lacking paragraph or Markdown notation cues), their cuts are highly arbitrary. Therefore, overlap improved performance in most tested configurations. This is particularly crucial for Character chunking, where the inherent risk of mid-word or mid-sentence text fragmentation makes an overlap essential for maintaining contextual coherence.

Finally, the optimization of the Max-Min Chunker confirmed the initial hypothesis regarding the algorithm’s threshold parameters. Due to the high thematic homogeneity of the administrative documents, utilizing the low dampening factor (c) and hard threshold from the original paper led to degenerate chunking behavior, where the algorithm grouped vast sections of text into unusable, oversized chunks. Conversely, setting these parameters too high triggered the opposite degeneration: almost every sentence was isolated into its own chunk, entirely defeating the purpose of semantic aggregation. The grid search successfully identified the algorithmic sweet spot, with RAG performance declining predictably when the thresholds were set either too high or too low.

6.3 Reranking Threshold

The empirical results largely confirm the initial hypothesis regarding cross-encoder similarity thresholding: chunkers characterized by a low average token count per chunk profit most from a dynamic cutoff score. Smaller, fine-grained chunks offer greater retrieval flexibility, as the system can retrieve varying numbers of highly targeted segments depending on query complexity. When paired with an appropriate similarity threshold, the retriever effectively filters out weakly scoring distractors, passing only high-confidence contexts to the Small Language Model (SLM) without forcing low-relevance chunks to fill an arbitrary context budget.

Across the evaluated strategies, a threshold benefit boundary between 100 and 200 tokens per chunk is hypothesized. Chunking methods producing average sizes below this threshold generally benefited from filtering at a 0.05 similarity cutoff. However, slight

nuances were observed near this boundary. Dynamic Token chunking (averaging 217.5 tokens per chunk) achieved slightly higher answer correctness with a low threshold of 0.05, whereas Character chunking on linearized text (averaging 121.6 tokens per chunk) performed marginally better without thresholding (0.00).

The primary exception to this pattern was the LLM chunker, which did not align with the general trend. Characterized by unusually large average chunk sizes that typically resulted in retrieving only a single chunk per query, its sensitivity to threshold adjustments was unpredicted. Rather than signaling a genuine structural interaction, this behavior likely highlights the inherent variance and evaluation noise associated with conducting preliminary hyperparameter tuning on the smaller 15-item subset.

Ultimately, the primary limitation preventing stricter thresholding from consistently improving performance across all configurations lies in the baseline reranking quality. The dense retriever and cross-encoder struggle to consistently rank the most relevant information within the top two or three candidates, the relevant chunks often only being in the middle of the retrieved contexts. Thus, essential evidence often receives comparatively lower similarity scores. Under these conditions, enforcing a non-zero similarity cutoff risks prematurely discarding necessary context before it reaches the generator. Consequently, while similarity thresholding holds clear theoretical and empirical advantages for fine-grained representations, its success remains inherently bottlenecked by the initial ranking precision of the retrieval pipeline. For a fine-tuned retriever and reranker, retrieval threshold can hold a much higher benefit.

6.4 Query Expansion

The results partly confirm the results from Jagerman et al. (2023). Chain of Thought performs best for more configurations than HyDE. However, also the approach without query preprocessing achieves good results, hinting to the fact that modern embedding models have improved coping with the query/document asymmetry.

6.5 Chunking and Preprocessing Baselines

Both baseline experiments confirm the need of a RAG pipeline with preprocessing and chunking. Without chunking, using RAG would be without purpose as there are only three documents. Thus, all three documents need to be given to the generator. The baseline experiments show that this causes the lost-in-the-middle phenomenon, degrading answer correctness.

6.6 Detailed Analysis of Markdown Chunking Strategies

6.6.1 Paragraph Chunking and Whole Table Chunking

Whole Table Chunking and Paragraph Chunking yielded very similar performance on Markdown documents due to their analogous segmentation mechanics. In Markdown-formatted text, double line breaks typically demarcate natural structural boundaries, causing standard Paragraph Chunking to split documents primarily at table or module borders. Conversely, Whole Table Chunking is designed to isolate structured elements as self-contained entities (as illustrated in Listing 8) while splitting unstructured prose at the paragraph level.

Both approaches achieved strong Context Recall, as the retriever generally only needs to identify and return one or two comprehensive chunks to ensure that the required evidence is fully present in the prompt. Whole Table Chunking holds a slight structural advantage over standard Paragraph Chunking because it systematically binds contextual metadata, such as headings and table captions, directly to the relevant tabular data. This structural coherence proved particularly beneficial for complex, distributed queries. For instance, when answering multi-entity prompts such as listing all courses required in the first semester, Whole Table Chunking outperformed all other evaluated chunking strategies. While other chunkers only managed partially correct answers due to context fragmentation across multiple chunks or overloaded chunks, Whole Table Chunking provided the entire semester schedule within a single, coherent context.

Despite these strengths, both Markdown-based strategies exhibited lower Faithfulness and marginally lower Answer Correctness compared to the top-performing chunkers operating on linearized text. Because Small Language Models (SLMs) with limited parameter counts are primarily optimized on natural language sequences, navigating dense, two-dimensional Markdown table syntax presents a notable challenge. When confronted with simple, fact-focused queries, the 2B generator frequently struggled to isolate specific numerical values or individual attributes from table cells, occasionally misassociating rows and columns. Consequently, while Whole Table and Paragraph Chunking excel at multi-attribute aggregation, their reliance on raw table structures introduces processing overhead that impairs the generator’s factual precision.

Listing 8: Example of Markdown structure in Markdown-converted fPO

```
### 1. Semester Bachelor Computerlinguistik
*Summe: 4 Module, 2 MP, 8 Veranstaltungen, 22 SWS, 30 LP, davon 1
  LP im PBB*

| Modul / Veranstaltung | MP | SL | LP | SWS |
| :--- | :--- | :--- | :--- | :--- |
| **Basismodul - Mathematische und logische Grundlagen** | **1 MP
  ** | **2 SL** | **12 LP** | **8 SWS** |
```

6.6.2 Character Chunking and Split Table Chunking

In contrast to the coarser segmentation approaches, both Character Chunking and Split Table Chunking performed poorly on Markdown documents, primarily due to internal table fragmentation. While language models generally exhibit proficiency in parsing standard Markdown tables when exposed to complete structures during pretraining, splitting tables internally severely disrupts the underlying syntax. This fragmentation heavily penalizes retrieval performance: because key contextual relations in structured formats are dispersed across multiple sub-table fragments, the dense retriever struggles to gather the necessary context, resulting in a substantially reduced Context Recall for both methods.

Surprisingly, this issue could not be mitigated by the structural contextualization implemented in Split Table Chunking. Although every Split Table fragment explicitly expanded abbreviations and prepended the relevant table caption, the SLM failed to cope with the splintered Markdown representation. While fine-grained chunks provided significant flexibility and comprehension benefits in linearized plain text, applying equivalent sub-document splits to structured Markdown yielded the exact opposite outcome. The

loss of complete row-column coordinates and headers prevented the generator from resolving cell-level relationships, dragging the performance of Split Table Chunking down to the same poor level as naive Character Chunking.

These structural breakdowns translated directly into degraded downstream generation, characterized by frequent query rejections, missed evidence within retrieved chunks, and pronounced grammatical, logical, and alignment errors (see Table 8). When receiving fragmented table contexts, the SLM often generated syntactically flawed phrasing, such as omitting auxiliary verbs (e.g., the *Bad Grammar* example for Split Table Chunking). More severely, broken context boundaries frequently caused the model to produce direct self-contradictions (e.g., the *Bad Logic* example for Character Chunking, asserting "Nein, du musst [...] eine Klausur/Prüfung absolvieren"). Finally, missing structural context sometimes led the generator to completely miss the user's intent: instead of evaluating the target module, the model retrieved unrelated fragments containing the keyword and listed irrelevant exam details from other courses (e.g., the *Missed Intent* example).

Table 8: Examples of the SLM failing to utilize split table content

Split Table (<i>Bad Grammar</i>) Query: Kann ich Einführung in die Programmierung mit Python auch im zweiten Semester belegen?	
Reference	Model Response
Der Studienverlaufsplan ist nur exemplarisch und nicht verpflichtend. Allerdings wird das Seminar "Einführung in die Programmierung mit Python" ausschließlich jedes Wintersemester angeboten. Da das zweite Semester in der Regel ein Sommersemester ist, ist eine reguläre Belegung dort nicht möglich.	Ja, die Einführung in die Programmierung mit Python kann im zweiten Semester belegen.
Character (<i>Bad Logic</i>) Query: Muss ich in Python eine Klausur/Prüfung absolvieren?	
Reference	Model Response
Ja, du musst im Modul "Python-Programmierung" eine Prüfung in Form einer Projektarbeit ablegen. Diese Modulprüfung ist jedoch erst im zweiten Fachsemester (Sommersemester) vorgesehen. Im ersten Semester musst du dafür lediglich Studienleistungen erbringen.	Nein, du musst in Python eine Klausur/Prüfung absolvieren.
Split Table (<i>Missed Intent</i>) Query: Gibt es in Machine Learning eine Klausur?	
Reference	Model Response
Nein, im Aufbaumodul "Maschinelles Lernen" wird keine Klausur geschrieben. Die Prüfungsform ist hier entweder eine Hausarbeit oder eine Studienarbeit, jeweils in Verbindung mit einem Programmierprojekt.	Ja, es gibt in den bereitgestellten Dokumenten Klausuren. Sie werden als "Klausur" oder "Modulprüfung (Klausur)" bezeichnet und finden in verschiedenen Modulen statt, wie z.B. im Aufbaumodul für Objektorientierte Programmierung (Pnr. 4510) oder im Basismodul für Python-Programmierung (Pnr. 2310).

6.7 Detailed Analysis of Text Chunking Strategies

6.7.1 Paragraph Chunking

For Paragraph Chunking, every chunk is quite large and factually dense. This makes it easy for the retriever to find the relevant chunks, resulting in high context recall. However, the SLM sometimes struggles with this type of input (Table 9). Especially specific information in the middle of a paragraph is often overlooked (lost-in-the-middle). This does not result in a low faithfulness score as it still mostly refers to information from the given context, just not the right information (Example A). In other cases, this results in the SLM rejecting the query (Example B). This results in a lower answer correctness.

Category	Example A: Partial / Incorrect Extraction	Example B: Erroneous Refusal / Hallucinated Absence
User Query	Welche Vorkenntnisse brauche ich für das Aufbaumodul Mathematische Linguistik?	Muss ich für den Python-Kurs Vorkenntnisse haben?
Context Extract	<i>"[...] Formale Teilnahmevoraussetzungen für das Aufbaumodul - Mathematische Linguistik sind keine, inhaltliche Teilnahmevoraussetzungen sind das Basismodul - Mathematische und logische Grundlagen und das Basismodul - Grundlagen der Computerlinguistik und formale Grammatikbeschreibung....]"</i>	<i>"[...] Für die Teilnahme am Basismodul - Python-Programmierung sind keine formalen oder inhaltlichen Teilnahmevoraussetzungen erforderlich....]"</i>
Reference Answer	Inhaltlich wird vorausgesetzt, dass du die Kurse aus dem "Basismodul - Mathematische und logische Grundlagen" sowie dem "Basismodul - Grundlagen der Computerlinguistik und formale Grammatikbeschreibung" kennst. Formale Zulassungsbeschränkungen gibt es für dieses Modul nicht.	Nein, für das Modul "Python-Programmierung" werden weder formale noch inhaltliche Vorkenntnisse vorausgesetzt.
SLM Response	Du brauchst keine Vorkenntnisse für das Aufbaumodul Mathematische Linguistik.	Dazu enthalten die bereitgestellten Dokumente keine Informationen.

Table 9: Information Extraction and Refusal Errors in SLM Outputs for Paragraph Chunking on text files

6.7.2 Character Chunking

Character Chunking has the known problem of cutting off sentences and even words. The loss of context is mitigated by using an overlap. However, these random cuts in the

beginning and in the end of a chunk result in worse vector representations and worse retrieval, thus worsening the context recall. Example B of Table 10 illustrates such a retrieval failure, where the retriever could not find the relevant chunk due to the information being in the middle of a chunk and the chunk starting mid-word of a different topic.

Having said that, Character Chunking still achieves the highest answer correctness. The shorter chunks are easier for the SLM to handle and find the information, resulting in the high faithfulness. Furthermore, the SLM was able to separate the chunks in its context more easily and locate the relevant information than for other chunkers. This is surprising, because normally, it is harder for language models to cope with in-word splits as they are trained on full-sentence natural language. An explanation could be that the in-word splits enable the SLM to separate the chunks from each other more easily, but this has to be looked into more. The SLM still sometimes struggles to locate the right information in the context. Boosting faithfulness is the fact that the SLM does not refuse to answer even in these cases, but cites some non-relevant context. However, not finding the relevant context happens less often than for other chunkers. In example A of Table 10, the SLM correctly identified the relevant information despite the relevant information being together in a chunk after two unrelated topics and the chunk starting mid-word. Furthermore, this was only the third retrieved chunk of seven, illustrating that chunk separation is easier for the SLM with Character Chunking.

6.7.3 Dynamic Token Chunking

Dynamic Token Chunking should be a good compromise between Paragraph Chunking and Character Chunking, having shorter chunks than Paragraph Chunking but not cutting off in the middle of sentences or words like Character Chunking. In some instances, this enabled the SLM to answer questions it could not for both Paragraph Chunking and Character Chunking (Table 11, Example A). However, its answer correctness was worse than for Character Chunking. While the more structured chunks made retrieval easier, resulting in a higher context recall, their larger size led to a higher risk of the SLM struggling to extract the answer from the information-dense sentences (Example B).

This dynamic highlights a critical trade-off in the RAG pipeline: while Dynamic Token Chunking successfully preserved syntactic integrity and boosted retrieval performance, it also increased the surrounding noise within each retrieved context. Consequently, the smaller language model, limited in its capacity to handle dense or distractor-heavy contexts, failed to reliably locate and extract the information from the larger text spans. Thus, this method should be tested with chunks smaller than 150 tokens.

6.7.4 LLM Chunking

LLM chunking, specifically LumberChunker, did not produce the expected results. In previous studies, this chunker had given the best results for in-document retrieval. Here, however, the chunker did not fit the linearized document format. As the paragraphs are relatively coarse-grained in relation to the document length, LumberChunker chunked the document into very large chunks by always putting several paragraphs into one chunk. This problem came also due to the homogeneity of the documents, which the chunker was not constructed for. This resulted in the average chunk size surpassing 750 tokens, and thus, there was always only one contextually dense chunk given to the LLM due to the context limit of 1500 tokens. This, of course, led to a low context recall, as even

Category	Example A: Context Chunk Extracted	Example B: Context Chunk Overlooked
User Query	Wie viele Prüfungen muss ich insgesamt ablegen?	Kann ich mein Praktikum machen, wann ich will?
Context Chunk	<i>"an einer weiterführenden Schule entspricht. Es gibt keine Angabe zu einem Propädeutikum im Profilbildungsbereich. Insgesamt sind 9 Modulprüfungen sowie die Bachelorarbeit zu absolvieren. Die Art und Inhalte der Module und Prüfungen im Bachelor Computerlinguistik umfassen folgende Basismodule: Mathematische und logische Grundlagen mit 8 SWS, 1 MP und 12 LP; Grundlagen der Computerlinguistik und formale Grammatikbeschreibung mit 8 SWS, 1 MP und 12 LP; und Python-Programmierung mit 8 SWS, 1 MP un"</i>	<i>"e Kenntnisse zum wissenschaftlichen Arbeiten, können eigene Forschungsfragen formulieren, diese angemessen eingrenzen und die zur Beantwortung notwendigen Schritte planen und umsetzen. Teilnahmevoraussetzungen für das Modul Praktikum & Orientierung sind im 4. Fachsemester oder höher, und inhaltlich das Basismodul - Mathematische und logische Grundlagen, das Basismodul - Grundlagen der Computerlinguistik und formale Grammatikbeschreibung und das Basismodul - Python-Programmierung. Für dieses Modu"</i>
Reference Answer	Im gesamten Bachelorstudium musst du 9 Modulprüfungen ablegen, zuzüglich der Bachelorarbeit.	Nein, du kannst den Zeitpunkt nicht völlig frei wählen. Als formale Teilnahmevoraussetzung für das Praktikums-Modul ist vorgeschrieben, dass du dich mindestens im 4. Fachsemester befinden musst.
SLM Response	Insgesamt sind 9 Modulprüfungen sowie die Bachelorarbeit zu absolvieren.	Dazu enthalten die bereitgestellten Dokumente keine Informationen.

Table 10: Examples of Successful Extraction versus Missing Context Retrieval for Character Chunking

Category	Example A: Successful Extraction	Example B: Information Lost / Extraction Failure
User Query	Gibt es eine Übung für den Kurs Mathematische Grundlagen?	Wann sollte ich Automatentheorie belegen?
Context Extract	”...Das Basismodul Mathematische und logische Grundlagen umfasst 1 MP, 2 SL, 12 LP und 8 SWS. Darin enthalten sind Einführung in die computationelle Logik (Seminar mit Übung, 4 LP, 4 SWS, Pnr. 2101), Mathematische Grundlagen (Seminar, 4 LP, 4 SWS, Pnr. 2102) und die Modulprüfung (Portfolio, 4 LP, Pnr. 2110)...”	”...Das Modul Aufbaumodul - Mathematische Linguistik (Modul-Nummer: 4200) hat einen Umfang von 6 ECTS-Leistungspunkten und einen Workload von 180 Stunden. Die Dauer dieses Aufbaumoduls beträgt 1 Semester. Die Lehrveranstaltungen für dieses Aufbaumodul sind Automaten-theorie & Formale Sprachen (Seminar) mit der Nummer 4201 und Übung zu Automatentheorie & Formale Sprachen (Übung) mit der Nummer 4202. Die Belegung für dieses Aufbaumodul erfordert die Teilnahme an Seminar (a) und Übung (b). Das empfohlene Studiensemester für dieses Aufbaumodul ist das 3. Fachsemester (Wintersemester) für beide Lehrveranstaltungen...”
Reference Answer	Nein, für den Kurs ”Mathematische Grundlagen” gibt es keine begleitende Übung. Es handelt sich dabei ausschließlich um ein Seminar.	Den Kurs Automatentheorie aus dem Modul Aufbaumodul – Mathematische Linguistik solltest du im dritten Semester belegen.
SLM Response	Nein, es gibt keine Übung für den Kurs Mathematische Grundlagen.	Dazu enthalten die bereitgestellten Dokumente keine Informationen.

Table 11: Examples of Successful Extraction versus Information Loss in Dynamic Token Chunking

in the cases where the one passage retrieved was right, more than one passage might be needed. This retrieval problem and the lost-in-the-middle phenomenon for the long chunk resulted in a low answer correctness.

6.7.5 Max-Min Chunking

Max-Min Chunking produced very compact segments, reinforcing the broader trade-off observed between retrieval performance and generation accuracy across different chunk sizes.

On the retrieval side, the reduced chunk size diminished context recall. Splitting text into smaller spans occasionally fragmented cohesive information across chunk boundaries, preventing the retriever from capturing the complete semantic context necessary for query matching or preventing the SLM from answering some questions. The ECTS example from Table 12 illustrates this issue. Specifically, the credit point calculation for the *Orientierungsseminar* (totalling 3 ECTS points) was severed across two separate chunks: Chunk A specified the contact hours (2 SWS / 30 hours / 1 ECTS point), whereas Chunk B detailed the independent self-study requirement (60 hours / 2 ECTS points). Because these complementary components were isolated from one another, the retrieval pipeline only returned Chunk A. Consequently, the SLM was deprived of the full mathematical context and generated an incomplete and factually incorrect answer (1 ECTS point) rather than aggregating the full workload.

Conversely, these smaller, focused chunks proved advantageous for the generation stage. Because the SLM received tightly scoped contexts with minimal surrounding noise, it was far less susceptible to the lost-in-the-middle phenomenon and could easily isolate relevant facts. Furthermore, the underlying semantic splitting logic ensured that information needed for straightforward queries remained contained within a single chunk in most cases. An example for a good split can be seen in Table 12. Here, the chunk about exam cancellation is completely thematically homogeneous and minimal. As a result of these chunks, despite the lower context recall, Max-Min Chunking achieved the second-highest answer correctness overall.

Thus, of the three more sophisticated chunking and RAG strategies from earlier papers tested or evaluated here, Max-Min Chunking is the only one with decent results. Both MiniRAG and LumberChunker failed to adapt to the homogeneous administrative documents. In the case of Max-Min Chunking, this only worked because of the hyperparameter fitting. Possibly, for this type of document and in combination with an SLM, more complex chunking and RAG methods can only outperform or even match deterministic chunking methods when tuned to the task. However, there needs to be more research done here.

Table 12: Comparison of Chunk Splitting Examples for Max-Min Chunking

Category	Text Content
Good Split (<i>Self-Contained Context</i>)	"Von einer Modulprüfung in Form einer Klausur oder mündlichen Prüfung kann sich die Kandidatin oder der Kandidat bis spätestens eine Woche vor dem jeweiligen Prüfungstermin abmelden."
Bad Split (<i>Fragmented ECTS Calculation</i>)	[Chunk A] "Die Kontaktzeit für das Orientierungsseminar (a) beträgt 2 SWS / 30 Stunden / 1 ECTS-Leistungspunkt." [Chunk B] "Das Selbststudium für das Orientierungsseminar (a) beträgt 60 Stunden / 2 ECTS-Leistungspunkte. Es gibt keine ECTS-Leistungspunkte für die Modulprüfung dieses Moduls."

6.8 Negative Rejection

While Character Chunking on linearized text achieved the highest overall answer correctness, it proved weak regarding negative rejection, exhibiting a high propensity for hallucinations. This deficiency primarily stems from the algorithm’s arbitrary text fragmentation, which arbitrarily cuts through sentences and groups disparate topics into single chunks. When confronted with unanswerable queries, the dense retriever pulls a highly fragmented, topically diverse set of text snippets. Lacking a reranking threshold to filter out low-confidence hits, the generator attempts to forcefully synthesize an answer from this noise. The first example in Table 13 illustrates the most severe case of this failure: when asked for the professors teaching the Machine Learning module, the model completely hallucinates names (e.g., “Prof. Dr. R. H.”) that do not appear anywhere in the provided documents.

In contrast, the Max-Min and Dynamic Token chunking strategies demonstrated significantly better negative rejection rates. By respecting sentence boundaries, these methods preserve semantic coherence and reduce the topical overlap within individual chunks. More importantly, both configurations utilized a reranking threshold (0.05). This threshold acts as a strict filter, aggressively discarding irrelevant context before it reaches the generator. Confronted with a query that has no factual basis in the text, the retriever simply passes very few or zero chunks to the Small Language Model (SLM), making it much easier for the model to correctly reject the prompt rather than invent an answer.

The large-scale Markdown strategies (Paragraph and Whole Table Chunking) showed moderate performance in negative rejection. The inherent structure of Markdown, particularly the presence of explicit table captions and clear module headers, generally assists the SLM in recognizing when a retrieved chunk is irrelevant to the prompt. However, the Markdown strategies inherently mix continuous text paragraphs from PO with tabular data, which can still occasionally mislead the model. As shown in the second example in Table 13 (generated using the Whole Table Chunker), the model correctly identified an entity name from a retrieved administrative chunk but falsely mapped it to the query’s intent, hallucinating that the university president (Anja Steinbeck) teaches the Machine Learning module.

Table 13: Examples of Hallucinations and Their Context Anchors for Negative Rejection

Query	Model Response	Hallucinated From (Context)
Welche Dozenten oder Professoren unterrichten das Aufbaumodul Maschinelles Lernen?	Basierend auf den bereitgestellten Dokumenten unterrichten folgende Dozenten oder Professoren das Aufbaumodul Maschinelles Lernen: • Prof. Dr. R. H. (nachweislich in Source 1) • Prof. Dr. M. H. (nachweislich in Source 2) • Prof. Dr. M. H. (nachweislich in Source 3, 5, 6, 8, 10–12)	[none]
Welche Dozenten oder Professoren unterrichten das Aufbaumodul Maschinelles Lernen?	Die Dozenten oder Professoren, die das Aufbaumodul Maschinelles Lernen unterrichten, sind: • Anja Steinbeck (Univ.-Prof. Dr. iur.)	[structured table chunk] *Anja Steinbeck (Univ.-Prof. Dr. iur.)* [structured table chunk]

6.9 Evaluation Analysis

In Table 14, six examples of the evaluation of Max-Min Chunking with their Answer Correctness score are shown. For most examples, the score is intuitively correct. The response for sample 5 is completely correct, giving a score very close to 1.0. For sample 4, the documents needed to answer the questions could not be found, resulting in a "no information found" response. Thus, the score is close to 0.0.

For partly correct answers, the scores were also mostly intuitively correct. In sample 3, the response only contains two of the four programming languages in the reference answer, thus the score is close to 0.5. The response of sample 1 is correct but missing the explanation, resulting in a score of 0.67.

There are some instances where the score is very strict or even wrong. In sample 2, the LLM correctly answers the question but is missing how many hours of courses have to be taken in Computer Science, which was not part of the query, but is given a score of 0.7. In sample 6, the response is reformulated, but completely correct and thus should be judged with 1.0 but instead is given a score of 0.72. However, this marks an exception and overall, the evaluations are mostly intuitive and correct.

Table 14: Evaluation Analysis

#	Query	Response	Reference Answer	Correctness
1	Muss ich in mathematische Grundlagen und in Logik eine Klausur schreiben?	Nein, es ist nicht erforderlich, in mathematische Grundlagen und in Logik eine Klausur zu schreiben.	Nein, du musst nicht in beiden Veranstaltungen eine Klausur schreiben. Für das gesamte "Basismodul - Mathematische und logische Grundlagen" ist nur eine einzige Modulprüfung in Form eines Portfolios vorgesehen.	0.6767
2	Kann ich für das interdisziplinäre Vertiefungsmodul nur linguistische Kurse belegen?	Nein, du kannst nicht nur linguistische Kurse belegen. Das interdisziplinäre Vertiefungsmodul erfordert neben Veranstaltungen aus der Computerlinguistik auch mindestens eine Veranstaltung aus der Informatik.	Nein, das ist nicht erlaubt. Das Modul verlangt zwingend, dass Veranstaltungen aus der Informatik im Umfang von mindestens 4 Semesterwochenstunden (SWS) belegt werden müssen.	0.7171
3	Welche Programmiersprachen lerne ich im Laufe des Studiums?	Im Laufe des Studiums lerne ich folgende Programmiersprachen: Python (im Basismodul) und R (im Modul Statistische Methoden in der Computerlinguistik).	In den Basismodulen lernst du ausführlich die Programmierung mit Python. Im Aufbaumodul "Objektorientierte Programmierung" wird eine objektorientierte Programmiersprache erlernt. Im Aufbaumodul "Logische Programmierung" wird in das logische Programmierparadigma anhand der Sprache Prolog eingeführt. Im Bereich Statistik kann die statistische Programmierung in einer geeigneten Sprache (beispielsweise R) erlernt werden.	0.5522

Continuation on the next page...

Table 14: (Continuation) Evaluation Analysis

#	Query	Response	Reference Answer	Correctness
4	Muss ich ein Praktikum machen?	Dazu enthalten die bereitgestellten Dokumente keine Informationen.	Nein, alternativ zu einem mindestens 6-wöchigen Berufsfeldpraktikum kann auch eine Projektarbeit absolviert werden, sofern diese einen einschlägig computerlinguistischen Inhalt aufweist.	0.0514
5	Wie oft darf ich eine nicht bestandene Modulprüfung wiederholen?	Eine nicht bestandene Modulprüfung kann zweimal wiederholt werden.	Eine nicht bestandene Modulprüfung kann insgesamt zweimal wiederholt werden.	0.9957
6	Wie viel Zeit haben die Lehrenden, um die Bewertung einer Klausur bekanntzugeben?	Die Bewertung einer Klausur wird nach spätestens acht Wochen bekannt gemacht.	Die Bewertung einer Klausur ist der Kandidatin oder dem Kandidaten nach spätestens acht Wochen bekannt zu machen.	0.7245

7 Limitations and Future Research

As this work focuses on preprocessing, chunking and query expansion, the optimization of other pipeline components is neglected. Especially the online part of the pipeline has a lot more potential for optimization. Even the maximum context size and the factor between top_k and top_n are optimizable in context of the rest of the pipeline. This could be done in an independent experiment or as part of a bigger grid search.

For the embedder, reranker, and generator, other models can be compared. For this work, only a few models were tested superficially, leaving this question open to future work. Especially for the generator, German-only models can be tested. Also larger models with a low quantization are an option. The retriever can even be combined with a sparse BM25 retriever. In previous work, this has outperformed a pure dense retriever (Sawarkar, Mangal, and Solanki 2024). However, especially (but not exclusively) when switching to hybrid retrieval, the results for chunking and preprocessing might have to be re-evaluated. For a similarly sized generative model, the results are more likely to hold up.

For improved retrieval results, the embedding model can be fine-tuned. As the topic is very specific and the number of documents is very limited, this could boost retrieval quality by a great margin. Furthermore, the query preprocessing can be more precise with fine-tuned embeddings.

Furthermore, the components optimized in the first four experiments (preprocessor, chunking parameters, context threshold and query expansion) can be further improved in a grid search. As the experiments were conducted sequentially due to evaluation resource constraints, there is no guarantee that this thesis found the best pipeline configuration. The best chunk size for RAG is for example heavily dependent on retrieval threshold, which was only investigated after the chunker experiment here. With more computational and monetary resources, future

research could improve the outcome by finding a better combination of components and settings. Furthermore, the small sample size for these experiments has an inherent risk of overfitting. A larger sample size could solve this problem.

For LLM chunking, a customized chunker for the documents could be built. This chunker should be based on sentences instead of paragraphs, as paragraphs are too coarse-grained here. Furthermore, the original Max-Min Chunking could potentially be improved by adding an overlap to the chunks, as the overlap improved results for other plain text chunkers with small chunk sizes.

For chatbot deployment, the repository can be made more performant using lazy imports, i. e., moving imports into methods. Thus, imports are only triggered when really needed. By using `vllm` with a locally saved model instead of the HuggingFace transformers library, there is an even greater potential gain in performance.

Concerning evaluation, more frameworks could be tested with even more robustness to differences in formulation. If the preprocessing/chunking setup is constant, traditional retrieval evaluation tools like `precision@k` or `recall@k` could be used. Otherwise, different tools for LLM-as-a-judge can be explored.

8 Conclusion

This thesis investigated the optimization of Retrieval-Augmented Generation (RAG) pipelines for Small Language Models (SLMs), using structured administrative documents from the Department of Computational Linguistics to develop a local, resource-efficient student advisory assistant. Because parameter-constrained models like Qwen3.5-2B are highly susceptible to lost-in-the-middle phenomena when processing dense contexts, all offline and some of the online components of the pipeline were systematically evaluated to maximize generation quality and factual reliability, focusing on chunking, document preprocessing and query expansion.

For evaluation, a dataset of 73 question-answer pairs of varying difficulty was created. These could be answered with the documents and served to evaluate answer correctness. Another 14 questions were created which could not be answered with the documents to test the negative rejection capability of the pipeline. This dataset can be used by future work and extended in case of new documents.

The code for the pipeline is highly modular and extensible. Both new documents and new components can be easily integrated. Thus, a future use in the department is easily feasible and even intended by the software architecture.

The empirical results validate the fundamental necessity of targeted document processing and chunking. Chunking combined with preprocessing substantially outperformed unchunked baselines, elevating answer correctness from 0.423 on raw text to over 0.570 on optimized configurations. Furthermore, even the lowest-performing chunkers outperformed the unchunked baseline.

For document conversion, prompting large models directly proved more effective than dedicated conversion frameworks. Task-specific prompts for Markdown extraction and text linearization consistently outperformed tools like Docling for Markdown, and a linearization strategy from existing literature. When comparing LLM-converted linearized text to Markdown, the top-performing strategies on LLM-converted linearized text achieved a slight edge in answer correctness over Markdown representations. However, the performance difference was narrow enough that Markdown remains a viable, structurally expressive alternative for future RAG pipelines.

The chunking experiments revealed distinct structural design principles across text formats. In structured Markdown documents, preserving complete module and table boundaries is critical: only Whole Table and Paragraph chunkers succeeded, whereas sub-table splitting caused syntax disruption, grammatical degeneration, and extraction failures. Plain text, conversely,

favored smaller chunk granularities. While Character chunking achieved the highest answer correctness when paired with an overlap to mitigate arbitrary cuts, it exhibited poor negative rejection by hallucinating non-existent facts when confronted with unanswerable queries. Max-Min chunking paired with a reranking threshold emerged as the most robust all-round strategy; when its hyperparameters were adapted to counter runaway aggregation in homogeneous administrative text, it delivered strong answer correctness while maintaining a high negative rejection rate of 0.857. Dynamic Token chunking demonstrated similar potential, indicating that fine-grained, sentence-aware chunkers are well suited for SLM generation provided that chunk sizes remain compact.

Both LLM Chunking and MiniRAG failed at the task at hand, while the hyperparameters for Max-Min Chunking had to be changed significantly in comparison to the original paper. This could mean that more complex chunking and RAG methods cannot be applied without fitting to RAG tasks with homogeneous, administrative documents, and that deterministic chunkers are more suited in this case.

In the online phase, applying a moderate cross-encoder similarity cutoff of 0.05 benefited fine-grained chunkers by filtering out noisy distractors on unambiguous queries, whereas larger chunks suffered from context loss. However, thresholding remains bottlenecked by baseline ranking precision, as critical evidence is not consistently positioned at the top of initial retrieval candidates. Regarding query expansion, Chain-of-Thought prompting slightly outperformed HyDE, but the unexpanded baseline remained competitive, underscoring the capacity of modern embedding models to bridge the asymmetry between queries and documents without mandatory query transformation.

The next step is to find an even more reliable evaluation framework. Then, more optimization possibilities for the RAG pipeline should be looked at. A fine-tuned hybrid retriever and fine-tuned reranker combined with optimizing the maximum context size given to the SLM could improve results significantly. Furthermore, different SLMs specific to German should be looked at and potentially fine-tuned.

Declaration of AI usage

I, Daniel Gelderblom, Matrikelnummer 3131057, hereby confirm that I have read and acknowledged the AI policy of the Institute for Linguistics, HHU (June 2025).

I hereby declare that for my thesis with the title Optimizing RAG Pipelines for Small Language Models, supervised by Prof. Dr. Wiebke Petersen and David Arps, I have used AI. The following list contains all tools that I have used as well as a short description of how and for what purpose these tools were used.

Name of the tools	version/date	Purpose
Google Gemini	gemini 3.1 pro and 3.5 flash	Finding literature (always checked and read thoroughly before use), sharpening ideas for the pipeline, suggesting qa-pairs for the evaluation dataset, Latex formatting, formulation from bulletpoints or reformulation for some paragraphs
Google Antigravity with Claude Sonnet	Antigravity IDE + Claude Sonnet 4.6	Implementation and documentation of pipeline and experiments, prompted with intended architecture and exact content

Eidesstattliche Versicherung

Ich, Daniel Gelderblom, versichere an Eides statt, dass die vorliegende Bachelorarbeit von mir selbstständig und ohne unzulässige fremde Hilfe unter Beachtung der "Grundsätze zur Sicherung guter wissenschaftlicher Praxis an der Heinrich-Heine-Universität Düsseldorf" erstellt worden ist.

Düsseldorf, 19.09.2026
Daniel Gelderblom

References

- Akram, Mohammad Kalim, Saba Sturua, Nastia Havriushenko, Quentin Herreros, Michael Günther, Maximilian Werk, and Han Xiao (2026). “jina-embeddings-v5-text: Task-Targeted Embedding Distillation”. In: *arXiv preprint arXiv:2602.15547*.
- Anthropic (Sept. 2024). *Contextual Retrieval*. <https://www.anthropic.com/news/contextual-retrieval>. Anthropic News & Announcements. Accessed: 2026-05-08.
- Bennani, Sofia and Charles Moslonka (2026). “A Systematic Analysis of Chunking Strategies for Reliable Question Answering”. In: *European Conference on Information Retrieval*. Springer, pp. 68–73.
- Brown, Tom, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. (2020). “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33, pp. 1877–1901.
- Chen, Si-An, Lesly Miculicich, Julian M Eisenschlos, Zifeng Wang, Zilong Wang, Yan-fei Chen, Yasuhisa Fujii, Hsuan-Tien Lin, Chen-Yu Lee, and Tomas Pfister (2024). “Tablerag: Million-token table understanding with language models”. In: *Advances in Neural Information Processing Systems* 37, pp. 74899–74921.
- Chen, Wei-Lin, Zhepei Wei, Xinyu Zhu, Shi Feng, and Yu Meng (2025). “Do LLM evaluators prefer themselves for a reason?”. In: *arXiv preprint arXiv:2504.03846*.
- Dai, Zihang, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov (2019). “Transformer-xl: Attentive language models beyond a fixed-length context”. In: *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 2978–2988.
- Deep Search Team (Aug. 2024). *Docling Technical Report*. Tech. rep. Version 1.0.0. DOI: 10.48550/arXiv.2408.09869. eprint: 2408.09869. URL: <https://arxiv.org/abs/2408.09869>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019). “Bert: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186.
- Douze, Matthijs, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou (2024). “The Faiss library”. In: arXiv: 2401.08281 [cs.LG].
- Duarte, André V, João DS Marques, Miguel Graça, Miguel Freire, Lei Li, and Arlindo L Oliveira (2024). “Lumberchunker: Long-form narrative document segmentation”. In: *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 6473–6486.
- Enevoldsen, Kenneth et al. (2025). “MMTEB: Massive Multilingual Text Embedding Benchmark”. In: *arXiv preprint arXiv:2502.13595*. DOI: 10.48550/arXiv.2502.13595. URL: <https://arxiv.org/abs/2502.13595>.
- Es, Shahul, Jithin James, Luis Espinosa Anke, and Steven Schockaert (2024). “Ragas: Automated evaluation of retrieval augmented generation”. In: *Proceedings of the 18th conference of the european chapter of the association for computational linguistics: system demonstrations*, pp. 150–158.
- Fan, Tianyu, Jingyuan Wang, Xubin Ren, and Chao Huang (2025). “Minirag: Towards extremely simple retrieval-augmented generation”. In: *arXiv preprint arXiv:2501.06713*.

- Gao, Luyu, Xueguang Ma, Jimmy Lin, and Jamie Callan (2023). “Precise zero-shot dense retrieval without relevance labels”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1762–1777.
- Günther, Michael, Isabelle Mohr, Daniel James Williams, Bo Wang, and Han Xiao (2024). “Late chunking: contextual chunk embeddings using long-context embedding models”. In: *arXiv preprint arXiv:2409.04701*.
- Guu, Kelvin, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang (2020). “Retrieval augmented language model pre-training”. In: *International conference on machine learning*. PMLR, pp. 3929–3938.
- Jagerman, Rolf, Honglei Zhuang, Zhen Qin, Xuanhui Wang, and Michael Bendersky (2023). “Query expansion by prompting large language models”. In: *arXiv preprint arXiv:2305.03653*.
- Kamradt, Greg (2024). *5 Levels of Text Splitting: A tutorial on different chunking strategies for RAG and LLM applications*. <https://github.com/FullStackRetrieval-com/RetrievalTutorials>. Accessed: 2026-05-25.
- Karpukhin, Vladimir, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih (2020). “Dense passage retrieval for open-domain question answering”. In: *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pp. 6769–6781.
- Kiss, Csaba, Marcell Nagy, and Péter Szilágyi (2025). “Max–Min semantic chunking of documents for RAG application”. In: *Discover Computing ; volume 28, issue 1 ; ISSN 2948-2992*. DOI: 10.1007/s10791-025-09638-7. URL: <https://doi.org/10.1007/s10791-025-09638-7>.
- Leng, Quinn, Jacob Portes, Sam Havens, Matei Zaharia, and Michael Carbin (2024). “Long context rag performance of large language models”. In: *arXiv preprint arXiv:2411.03538*.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. (2020). “Retrieval-augmented generation for knowledge-intensive nlp tasks”. In: *Advances in neural information processing systems* 33, pp. 9459–9474.
- Liu, Nelson F, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang (2024). “Lost in the middle: How language models use long contexts”. In: *Transactions of the association for computational linguistics* 12, pp. 157–173.
- Liu, Yang, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu (2023). “G-eval: NLG evaluation using gpt-4 with better human alignment”. In: *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 2511–2522.
- Min, Dehai, Nan Hu, Rihui Jin, Nuo Lin, Jiaoyan Chen, Yongrui Chen, Yu Li, Guilin Qi, Yun Li, Nijun Li, et al. (2024). “Exploring the impact of table-to-text methods on augmenting llm-based question answering with domain hybrid data”. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pp. 464–482.
- Muennighoff, Niklas, Nouamane Tazi, Loïc Magne, and Nils Reimers (2022). “MTEB: Massive Text Embedding Benchmark”. In: *arXiv preprint arXiv:2210.07316*. DOI: 10.48550/ARXIV.2210.07316. URL: <https://arxiv.org/abs/2210.07316>.
- Nogueira, Rodrigo and Kyunghyun Cho (2019). “Passage Re-ranking with BERT”. In: *arXiv preprint arXiv:1901.04085*.

- Qu, Renyi, Ruixuan Tu, and Forrest Bao (2025). “Is semantic chunking worth the computational cost?” In: *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 2155–2177.
- Qwen (2026). *Qwen3.5-2B*. <https://huggingface.co/Qwen/Qwen3.5-2B>. Model repository.
- Sawarkar, Kunal, Abhilasha Mangal, and Shivam Raj Solanki (2024). “Blended rag: Improving rag (retriever-augmented generation) accuracy with semantic search and hybrid query-based retrievers”. In: *2024 IEEE 7th international conference on multimedia information processing and retrieval (MIPR)*. IEEE, pp. 155–161.
- Si, Jacob, Mike Qu, Michelle Lee, and Yingzhen Li (2025). “TabRAG: Tabular Document Retrieval via Structured Language Representations”. In: *arXiv preprint arXiv:2511.06582*.
- Wang, Feng, Yuqing Li, and Han Xiao (2025). *jina-reranker-v3: Last but Not Late Interaction for Document Reranking*. arXiv: 2509.25085 [cs.CL]. URL: <https://arxiv.org/abs/2509.25085>.
- Yu, Xiaohan, Pu Jian, and Chong Chen (2025). “Tablerag: A retrieval augmented generation framework for heterogeneous document reasoning”. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 14074–14093.
- Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. (2023). “Judging llm-as-a-judge with mt-bench and chatbot arena”. In: *Advances in neural information processing systems* 36, pp. 46595–46623.
- Zhou, Yongjie, Shuai Wang, Bevan Koopman, and Guido Zuccon (2026). “Beyond Chunk-Then-Embed: A Comprehensive Taxonomy and Evaluation of Document Chunking Strategies for Information Retrieval”. In: *arXiv preprint arXiv:2602.16974*.